

SYSTEM DESIGN GROUP PROJECT · 2026

# AyuSangh

## Digital Healthcare Discovery & Institutional Rating Platform

*Problem Statement & Software Requirements Specification*

|               |                                                                         |
|---------------|-------------------------------------------------------------------------|
| Project Name  | AyuSangh — Digital Healthcare Discovery & Institutional Rating Platform |
| Document Type | Problem Statement & Software Requirements Specification (SRS)           |
| Version       | 1.0                                                                     |
| Date          | 2026                                                                    |
| Course        | System Design — Group Project                                           |
| Architecture  | Modular Monolith (Microservices-ready)                                  |
| Tech Stack    | NestJS · TypeScript · PostgreSQL · Prisma · Next.js                     |

# Document 1 — Problem Statement

## 1.1 Background & Context

India has over 70,000 hospitals, 150,000+ nursing homes, 100,000+ diagnostic centres, and millions of clinics spread across its cities and towns. Yet a patient in Noida, Patna, or Surat who needs to find a trustworthy diagnostic lab for an MRI scan or a reliable nursing home for a post-surgery stay has almost no structured way to do so.

Existing platforms like Practo, 1mg, and JustDial have partially addressed this — but all of them are built around the doctor as the primary entity. Hospitals, labs, nursing homes, and clinics are treated as secondary context to a doctor listing. This creates a fundamental mismatch with how patients actually make healthcare decisions.

## 1.2 The Core Problem

The Indian healthcare discovery market suffers from three compounding problems:

### Problem 1 — Institution invisibility

Hospitals, diagnostic labs, and nursing homes have no structured digital identity. A hospital's profile on existing platforms is just a location pin attached to a doctor listing. Patients cannot find information about a hospital's services, cleanliness standards, infrastructure, or actual patient experiences in one place.

### Problem 2 — Doctor-first model does not reflect real-world decision making

When a patient needs an MRI, they do not think 'I need Dr. Sharma.' They think 'I need a diagnostic centre with an MRI machine that is nearby, affordable, and has good patient reviews.' When a family needs a nursing home for post-operative care, they are choosing an institution — not an individual doctor. No existing platform models this reality.

### Problem 3 — Tier 2 and Tier 3 city healthcare is completely unrepresented

Practo and 1mg have strong coverage in metro cities but minimal data for cities like Meerut, Agra, Kanpur, Patna, and Indore — where millions of patients make healthcare decisions based purely on word-of-mouth or random Google searches. This information asymmetry leads to poor choices, overpaying, and wasted time.

## 1.3 Evidence of the Problem

| Platform | What it does well                                  | What is missing                                                      |
|----------|----------------------------------------------------|----------------------------------------------------------------------|
| Practo   | Doctor discovery, clinic booking, teleconsultation | No hospital profile, no lab comparison, no institution-level reviews |

|                   |                                      |                                                                     |
|-------------------|--------------------------------------|---------------------------------------------------------------------|
| 1mg / Tata Health | Medicine delivery, basic lab booking | No institution rating, no multi-service search                      |
| JustDial          | Business listings with contact info  | No verified reviews, no healthcare-specific filters                 |
| Google Maps       | Location + basic rating              | No healthcare context, no service-based search, fake reviews common |

## 1.4 Who Is Affected

- Patients seeking diagnostic tests — cannot compare prices, turnaround times, or NABL certification
- Families choosing nursing homes — no verified reviews or infrastructure information
- Patients in emergencies — no quick way to find the nearest hospital with 24/7 ICU availability
- Patients in Tier 2/3 cities — no structured healthcare discovery at all
- Institutions themselves — no platform to publish their services, certifications, and photos professionally

## 1.5 Proposed Solution

AyuSangh is a Healthcare Institutional Discovery and Rating Platform where healthcare institutions — not individual doctors — are the primary entity. Patients can search for hospitals, nursing homes, diagnostic centres, and clinics by location and service need, read multi-dimensional patient reviews, and access booking contact links. Institutions can manage their own profiles, publish their services and certifications, and respond to reviews.

This is not another Practo clone. It is a fundamentally different data model: Institution is the root entity. Doctors are associated entities that belong to institutions. This reflects real-world permanence — doctors move, institutions stay.

## 1.6 Solution Differentiators

- Institution-first architecture — the root entity is the institution, not the doctor
- Multi-dimensional rating system — patients rate Cleanliness, Staff Behaviour, Wait Time, and Overall separately
- Three-voice institution profile — platform-generated summary, institution-written about section, and patient reviews
- Service-based search — patients search by what they need (MRI, blood test, maternity care), not by institution name
- Designed for Indian Tier 2/3 cities — not just metro-centric

## Document 2 — Software Requirements Specification (SRS)

### 2.1 System Scope

AyuSangh is a web-based platform that enables patients to discover, evaluate, and access healthcare institutions across India. The system manages institution profiles, doctor associations, patient-submitted reviews, multi-category ratings, location-based search, and role-based user access.

The system is built as a Modular Monolith using NestJS, with architecture designed for future microservice extraction. The frontend is a Next.js application consuming a REST API.

### 2.2 Actors & Roles

| Actor             | Description                                                                                   | Access Level         |
|-------------------|-----------------------------------------------------------------------------------------------|----------------------|
| Guest User        | Unauthenticated visitor — can browse, search, and read reviews                                | Read-only public     |
| Patient           | Registered user — can submit reviews, save favourites, manage profile                         | Read + Write reviews |
| Doctor            | Registered doctor — can manage own profile and institutional affiliation                      | Own profile only     |
| Institution Admin | Hospital/lab/clinic representative — manages institution profile, photos, responds to reviews | Own institution only |
| Platform Admin    | Internal team — manages all data, handles verification, moderation                            | Full system access   |

### 2.3 Functional Requirements

#### FR-01 Authentication & Authorisation

- FR-01.1 Users can register with email and password
- FR-01.2 Passwords are hashed using bcrypt before storage
- FR-01.3 Authentication uses stateless JWT tokens
- FR-01.4 Role-Based Access Control (RBAC) enforced on all protected endpoints
- FR-01.5 Refresh token strategy for session continuity

#### FR-02 Institution Management

- FR-02.1 Platform supports four institution types: Hospital, Nursing Home, Diagnostic Centre, Clinic

- FR-02.2 Each institution has: name, type, description, address, city, pincode, phone, photos, services list, booking link
- FR-02.3 Institution Admin can create and update their institution profile
- FR-02.4 Institution Admin can upload photos (stored via Cloudinary)
- FR-02.5 Institution Admin can add a booking redirect link
- FR-02.6 Platform Admin can verify and moderate institution listings

### **FR-03 Doctor Management**

- FR-03.1 Doctors can create profiles with: name, specialty, experience, qualifications, consultation fee
- FR-03.2 Doctors can associate themselves with one or more institutions
- FR-03.3 Institution profile displays list of associated doctors
- FR-03.4 Patients can search for doctors by specialty and institution

### **FR-04 Review & Rating System**

- FR-04.1 Registered patients can submit one review per institution
- FR-04.2 Each review contains: written text, and ratings for Overall, Cleanliness, Staff Behaviour, Wait Time
- FR-04.3 All ratings use a 1–5 scale
- FR-04.4 Institution average ratings are auto-calculated using Strategy Pattern
- FR-04.5 Institution Admin can reply to patient reviews
- FR-04.6 Patients can edit or delete their own reviews
- FR-04.7 Platform Admin can moderate and remove reviews

### **FR-05 Search & Discovery**

- FR-05.1 Users can search institutions by city, pincode, or institution name
- FR-05.2 Users can filter results by institution type, rating, and open status
- FR-05.3 Search results are sorted by rating (descending) by default
- FR-05.4 Full-text search on institution name and services (PostgreSQL FTS)
- FR-05.5 Guest users have full access to search and profile viewing

### **FR-06 User Dashboard**

- FR-06.1 Patients can view and edit their submitted reviews
- FR-06.2 Patients can save institutions to a favourites list
- FR-06.3 Institution Admin can view all reviews for their institution
- FR-06.4 Institution Admin can manage doctor associations

## **2.4 Non-Functional Requirements**

| Category        | Requirement                                         | Target Metric                            |
|-----------------|-----------------------------------------------------|------------------------------------------|
| Performance     | Search API response time                            | < 300ms for 95th percentile              |
| Scalability     | Architecture must support horizontal scaling        | Microservice-extractable modules         |
| Security        | All passwords hashed, JWT expiry enforced           | bcrypt + 1hr access token                |
| Availability    | Platform available during demo and evaluation       | 99% uptime on hosted deployment          |
| Maintainability | Each module independently testable                  | Controller/Service/Repository separation |
| Usability       | Guest user can find an institution in under 3 steps | Search → Results → Profile               |
| Data Integrity  | All foreign key constraints enforced at DB level    | PostgreSQL FK constraints                |
| Caching         | Frequently accessed institution profiles cached     | Redis (planned v2)                       |

## 2.5 System Constraints

- Backend language is TypeScript exclusively — no JavaScript files in the codebase
- pnpm is the only permitted package manager — npm usage is rejected
- All database interaction goes through Prisma ORM — no raw SQL except for complex analytics
- Business logic must reside in Service layer only — controllers handle HTTP only
- No use of 'any' type in TypeScript

## 2.6 Out of Scope (MVP / Capstone Version)

The following features are architecturally considered but not implemented in the capstone submission:

- In-app appointment booking engine (booking link redirect is implemented instead)
- Prescription upload and verification
- Insurance integration
- Real-time bed or ICU availability tracking
- AI-based recommendation engine
- Mobile application
- Payment gateway integration

## 2.7 Assumptions

1. Institution data is manually seeded for the demo — no live hospital API integration

2. All uploaded images are hosted on Cloudinary — no local file storage
3. The platform targets web browsers only in this version
4. Reviews are text-based only — no photo upload in reviews for capstone
5. Email verification is mocked for demo purposes — real SMTP in v2

## Document 3 — Design Patterns & SOLID Justification

### 3.1 Design Patterns Used

#### Pattern 1 — Repository Pattern

Every module has a dedicated repository class (e.g. InstitutionRepository, ReviewRepository) that abstracts all database interactions from the service layer. Services never call Prisma directly.

- Decouples business logic from database technology
- Allows mocking in unit tests without a real database
- Consistent with Open/Closed Principle — swap DB without touching service code

Example: InstitutionService calls InstitutionRepository.findByCity(city). The repository handles the Prisma query. The service does not know or care how the data is fetched.

#### Pattern 2 — Strategy Pattern

The rating calculation system uses the Strategy Pattern. Different institution types may use different scoring algorithms. A common IRatingStrategy interface defines the calculateAverage() method. Concrete strategies (HospitalRatingStrategy, LabRatingStrategy) implement this interface.

- Allows adding new rating logic without modifying existing code (Open/Closed)
- Each strategy is independently testable
- RatingContext class selects the correct strategy at runtime

#### Pattern 3 — Service Layer Pattern

All business logic is contained exclusively within service classes. Controllers are thin HTTP adapters. This ensures single responsibility and makes business logic independently testable.

#### Pattern 4 — Factory Pattern (optional, planned)

InstitutionFactory creates Institution objects of different types (Hospital, Lab, Clinic, NursingHome) from a common input shape. This centralises object creation and supports the Open/Closed Principle for future institution types.

### 3.2 SOLID Principles Implementation

| Principle             | How it is applied                                                                         | Where in code      |
|-----------------------|-------------------------------------------------------------------------------------------|--------------------|
| Single Responsibility | Each class has exactly one reason to change. AuthService handles only auth. ReviewService | All module classes |



|                       |                                                                                                                                                                              |                                    |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
|                       | handles only reviews. Controllers handle only HTTP.                                                                                                                          |                                    |
| Open / Closed         | New institution types added via enum + factory — no existing code modified. New rating strategies added as new classes implementing IRatingStrategy.                         | InstitutionFactory, RatingStrategy |
| Liskov Substitution   | All institution types implement the same IIstitution interface and are fully substitutable. Any code accepting IIstitution works with Hospital, Lab, Clinic, or NursingHome. | Institution module                 |
| Interface Segregation | IIstitutionRepository and IReviewRepository are separate interfaces. No module is forced to implement methods it does not use.                                               | All repository interfaces          |
| Dependency Inversion  | Controllers depend on service interfaces, not concrete classes. Services depend on repository interfaces. NestJS DI container injects concrete implementations.              | All modules via NestJS DI          |

### 3.3 OOP Concepts Implementation

| OOP Concept   | How it is applied                                                                                                                     | Example                                        |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| Encapsulation | Module internals hidden behind service interfaces. Internal Prisma calls never exposed to controllers.                                | InstitutionService wraps InstitutionRepository |
| Abstraction   | Repository interfaces abstract database implementation. Services interact with abstractions.                                          | IIstitutionRepository interface                |
| Inheritance   | BaseEntity class with id, createdAt, updatedAt extended by all entities. Base exception filters shared via extension.                 | BaseEntity, BaseException                      |
| Polymorphism  | IRatingStrategy implemented differently by HospitalRatingStrategy and LabRatingStrategy. RatingContext calls the same method on both. | Strategy Pattern implementation                |

### 3.4 Architecture Decision — Why Modular Monolith

A Modular Monolith was chosen over a full Microservices architecture for the following reasons:

- Team size (5 developers) — microservices introduce operational overhead that exceeds team capacity at this stage

- Development speed — shared database and single deployment simplify the build process
- Capstone evaluation — evaluators can run the system locally without Docker orchestration complexity
- Future readiness — each NestJS module is designed to be extracted into an independent service as the platform scales

The architecture is not a compromise — it is the correct choice for this phase. Every module is independently testable, has clean boundaries, and can be deployed separately when operational requirements demand it.

*End of Document*

AyuSangh · System Design Group Project 2026 · Version 1.0